The Metatheory of Crush's Certified Lowering Pipeline

lean-crush

August 2026

Abstract

Crush translates a supported fragment of Lean propositions into SMT commands. This document explains the mathematical models used for that translation, the total flattened defunctionalization algorithm, the representation of typed first-order formulas by the untyped production SMT syntax, and the precise soundness theorem proved in Lean. It is written as a guide to the proof: every language and semantic object is introduced before its correctness theorem, and the final roadmap separates completed results from remaining trust boundaries. The theorem justifies the reified higher-order sentence and the exact commands produced by the proved VCG route; it does not assume that an external solver's answer is correct and does not assign a denotation to arbitrary `Lean.Expr`.

Contents

1 Reader's guide

Contents of this section. The question answered by the formalization; proof stages; notation and scope.

1.1 What is being proved

The formalization studies this pipeline:

The first arrow is a partial recognizer: unsupported Lean syntax is rejected. The second and third arrows are total on their typed inputs. The principal semantic conclusion is:

> If the exact raw commands returned by the proved VCG route are semantically unsatisfiable, then the reified higher-order source sentence is unsatisfiable.

The proof is by countermodel construction. Starting from any model of the higher-order sentence, Lean constructs a model of the flattened first-order theory, then constructs a relational SMT model satisfying the exact encoded commands. Taking the contrapositive of those two constructions yields the desired reflection of unsatisfiability.

1.2 Proof stages

The argument is organized into five stages:

1. Define an intrinsically typed higher-order source language and its ordinary function-space semantics.
2. Define an intrinsically typed first-order target whose function-value sorts are opaque, then give total flattened defunctionalization.
3. Interpret the target in a canonical model built from a source model and prove term preservation plus validity of every generated obligation.
4. Encode the typed target theory into raw SMT syntax and prove that every target model induces a model of the exact command sequence.
5. Retain structural evidence for Lean reification and exact stateful VCG execution, while marking the remaining trusted paths explicitly.

1.3 Metavariables and notation

The following symbols are used consistently throughout the document:

e, f, a	source or target terms; f is usually function-valued,
φ, ψ	Boolean formulas; a closed formula is a sentence,
β	an opaque source base-sort name,
$\alpha, \gamma, \sigma, \tau, \upsilon$	source types,
κ	a first-order sort,
Γ, Δ	typing contexts,
Σ	a typed signature,
T	a typed first-order theory,
M, N	semantic models,
ρ	a source valuation or typed renaming,
ν	a target valuation,
v, w	individual semantic values,
$\mathcal{E}$	a concrete FO-to-SMT encoding choice,
C	an ordered array of raw SMT commands.

Subscripts s, t, and q mean source, typed target, and raw SMT model, respectively. The basic semantic notation is

$[\![e]\!]_{M,\rho}$	denotation of e in M under ρ,
$M \models \varphi$	M satisfies the closed formula φ,
$M \models^{\mathsf{T}} T$	M satisfies every formula in theory T,
$\lfloor \tau \rfloor$	first-order erasure of source type τ,
$\lfloor \Gamma \rfloor^{\star}$	pointwise erasure of source context Γ,
$\mathcal{D}[\![e]\!]$	the classic unary reference translation,
$\mathcal{F}[\![e]\!]$	the total flattened translation result,
$\mathcal{A}_{\mathcal{E}}[\![t]\!]$	raw SMT term encoding of typed FO term t.

The two translation brackets denote syntax-to-syntax transformations, whereas $[\![e]\!]_{M,\rho}$ denotes a semantic value. The notation mirrors the scoped Lean notation and the flattened/SMT encoder modules. [1]

2 The intrinsically typed higher-order source

Contents of this section. Types, contexts, terms, and ordinary higher-order semantics.

2.1 Types, contexts, and signatures

Definition 2.1 (Source types)**.** *Let β range over opaque base-sort names. Source types are generated by*

$$\tau ::= \mathsf{Bool} \mid \mathsf{Base}(\beta) \mid \tau \to \sigma.$$

Arrows are nondependent and associate to the right. A context $\Gamma = [\tau_0, \ldots, \tau_{n-1}]$ records local binders with the newest binder at the head. We write Γ, τ for extension by a new binder of type τ; this is stored as $\tau :: \Gamma$ in Lean. A signature Σ is a list of the types of global constants. [2]

[1] Lean formalization: `Crush/Metatheory/Notation.lean`, `Defunctionalization/Flattened/Translate.lean`, and `SMT/Representation.lean`.

[2] Lean formalization: `Crush/Metatheory/HO/Core.lean`.

We write $\Gamma \ni \tau$ for the intrinsically typed lookup judgment. Its inhabitants are de Bruijn references, generated by:

$$\frac{}{Z : \Gamma, \tau \ni \tau} \qquad \frac{i : \Gamma \ni \tau}{S\, i : \Gamma, \sigma \ni \tau}\ .$$

The judgment is intrinsically witnessed: its derivation simultaneously records the position and proves the type at that position. The witness Z denotes index zero and $S\, i$ denotes the successor of the index denoted by i. Forgetting the derivation yields the natural-number index used when collecting free variables. Constants use the same typed-reference representation in Σ. Therefore a variable or constant with the wrong type cannot be constructed.

For $i : \Gamma \ni \tau$, write x_i for the local-variable term selected by i. For $k : \Sigma \ni \tau$, write c_k for the global-constant term selected by k from the signature. The subscripts are typed lookup witnesses, not variable or constant names.

Definition 2.2 (Source terms)**.** *The intrinsic judgment $\Sigma; \Gamma \vdash e : \tau$ has variables, constants, Boolean literals and connectives, same-type equality, lambda, application, and universal and existential quantification:*

$$\frac{i : \Gamma \ni \tau}{\Sigma; \Gamma \vdash x_i : \tau} \qquad \frac{k : \Sigma \ni \tau}{\Sigma; \Gamma \vdash c_k : \tau} \qquad \frac{\Sigma; \Gamma, \tau \vdash e : \sigma}{\Sigma; \Gamma \vdash \lambda x{:}\tau.e : \tau \to \sigma} \qquad \frac{\Sigma; \Gamma \vdash f : \tau \to \sigma \qquad \Sigma; \Gamma \vdash a : \tau}{\Sigma; \Gamma \vdash f\, a : \sigma}$$

$$\frac{\Sigma; \Gamma \vdash e_1 : \tau \qquad \Sigma; \Gamma \vdash e_2 : \tau}{\Sigma; \Gamma \vdash e_1 = e_2 : \mathsf{Bool}} \qquad \frac{\Sigma; \Gamma, \tau \vdash \varphi : \mathsf{Bool} \qquad Q \in \{\forall, \exists\}}{\Sigma; \Gamma \vdash Qx{:}\tau.\varphi : \mathsf{Bool}}.$$

The omitted rules are the standard typed Boolean rules. A formula is a term of type Bool*; a sentence is a formula in the empty context.* [3]

Quantifiers may range over arrow types. Thus the modeled language is genuinely higher-order even though its arrows are nondependent.

2.2 Source models

Definition 2.3 (Source type denotation)**.** *A carrier family B assigns a nonempty type $B(\beta)$ to each opaque base sort. Its extension to source types is*

$$[\![\mathsf{Bool}]\!]_B = \mathsf{Prop}, \qquad [\![\mathsf{Base}(\beta)]\!]_B = B(\beta), \qquad [\![\tau \to \sigma]\!]_B = [\![\tau]\!]_B \to [\![\sigma]\!]_B.$$

Thus the only choices in B are the carriers assigned to opaque base sorts.

Definition 2.4 (Source model and valuation)**.** *A source model M_s consists of a carrier family B and an interpretation $c_k^{M_s} \in [\![\tau]\!]_B$ for each $k : \Sigma \ni \tau$. A valuation ρ for Γ assigns a value in $[\![\tau]\!]_B$ to every $i : \Gamma \ni \tau$. We write $\rho[v]$ for extension by a new head value v.*

Definition 2.5 (Source denotation)**.** *Denotation is structural. Variables use ρ, constants use M_s, and the Boolean forms have their ordinary meanings. The higher-order clauses are*

$$\begin{aligned}
[\![\lambda x{:}\tau.e]\!]_{M_s,\rho} &= \lambda v.[\![e]\!]_{M_s,\rho[v]},\\
[\![f\, a]\!]_{M_s,\rho} &= [\![f]\!]_{M_s,\rho}([\![a]\!]_{M_s,\rho}),\\
[\![\forall x{:}\tau.\varphi]\!]_{M_s,\rho} &= \forall v \in [\![\tau]\!]_B, [\![\varphi]\!]_{M_s,\rho[v]},\\
[\![\exists x{:}\tau.\varphi]\!]_{M_s,\rho} &= \exists v \in [\![\tau]\!]_B, [\![\varphi]\!]_{M_s,\rho[v]}.
\end{aligned}$$

[3] Lean formalization: `Crush/Metatheory/HO/Core.lean`.

For a sentence φ, $M_s \models \varphi$ means evaluation under the empty valuation.[4]

Definition 2.6 (Source unsatisfiability)**.** *A source sentence is semantically unsatisfiable when no source model satisfies it:*

$$\mathsf{Unsat}_{\mathsf{HO}}(\varphi) \quad \Longleftrightarrow \quad \forall M_s.\, \neg(M_s \models \varphi).$$

This definition is semantic: it quantifies over the mathematical models just introduced.

3 The intrinsically typed first-order target

Contents of this section. Erased sorts, abstract symbol families, target terms, and target models.

Definition 3.1 (First-order sorts and erasure)**.** *Target sorts are*

$$\kappa ::= \mathsf{Bool} \mid \mathsf{Base}(\beta) \mid \mathsf{Fn}(\tau, \sigma).$$

Erasure is defined by

$$\lfloor \mathsf{Bool} \rfloor = \mathsf{Bool}, \qquad \lfloor \mathsf{Base}(\beta) \rfloor = \mathsf{Base}(\beta), \qquad \lfloor \tau \to \sigma \rfloor = \mathsf{Fn}(\tau, \sigma).$$

The last sort is opaque: it is not a function space in an arbitrary target model. Context erasure maps every entry pointwise. [5]

Definition 3.2 (Declarations and symbol families)**.** *A declaration*

$$d = (\kappa_1, \ldots, \kappa_n; \kappa)$$

records an ordered argument telescope $d.\mathsf{args} = [\kappa_1, \ldots, \kappa_n]$ and a result sort $d.\mathsf{result} = \kappa$. Let Decl *be the type of all such declarations. A* symbol family *is a declaration-indexed family of types*

$$S : \mathsf{Decl} \longrightarrow \mathbf{Type}, \qquad d \longmapsto S(d).$$

An inhabitant $p : S(d)$ is one symbol identity whose declaration is exactly d. The fiber $S(d)$ may contain no symbols, one symbol, or several distinct symbols with the same argument and result sorts. Thus the declaration is part of a symbol's type, while its identity remains separate from its declaration. [6]

Definition 3.3 (Typed family arguments and symbol application)**.** *Terms and heterogeneous argument vectors are defined mutually. For a context Γ, the argument-vector family follows the declaration telescope:*

$$\mathsf{Args}_S(\Gamma, []) = \{()\},$$
$$\mathsf{Args}_S(\Gamma, \kappa_1 :: \bar{\kappa}) = \{t \mid S; \Gamma \vdash t : \kappa_1\} \times \mathsf{Args}_S(\Gamma, \bar{\kappa}).$$

The only general symbol-application rule is

$$\frac{p : S(d) \qquad \bar{t} : \mathsf{Args}_S(\Gamma, d.\mathsf{args})}{S; \Gamma \vdash p(\bar{t}) : d.\mathsf{result}}.$$

Consequently the result sort and the complete ordered list of argument sorts are determined by the index d. A missing, extra, or incorrectly sorted argument cannot be represented. In particular, symbols are always fully applied; partial application is not a target-language operation.

[4] Lean formalization: `Crush/Metatheory/HO/Semantics.lean`.
[5] Lean formalization: `Crush/Metatheory/FO/Core.lean`.
[6] Lean formalization: `Crush/Metatheory/FO/Family.lean`.

For example, let $d = (\kappa_1, \kappa_2; \kappa)$ and let $p, q : S(d)$ be distinct symbols. Given $S; \Gamma \vdash u : \kappa_1$ and $S; \Gamma \vdash v : \kappa_2$, both $p(u, v)$ and $q(u, v)$ have sort κ, but they remain different terms. Neither $p(u)$ nor $p(v, u)$ is constructible.

Definition 3.4 (Target terms)**.** *For a symbol family S, the intrinsic judgment $S; \Gamma \vdash t : \kappa$ contains variables, the fully applied family symbols above, Boolean literals and connectives, same-sort equality, and first-order quantifiers. It contains neither lambda nor a distinguished higher-order application constructor. A function value is merely an element of an opaque $\mathsf{Fn}(\tau, \sigma)$ carrier; applying that value later requires an ordinary family symbol with the appropriate declaration.* [7]

Why translation uses a family. The finite target syntax also exists: a finite signature is a list $\Xi = [d_0, \ldots, d_m]$, and a symbol occurrence carries a typed position showing that some d_j is its declaration. That form is useful after allocation, but inconvenient during recursive translation. A recursive branch may discover a new closure or application symbol; combining two branches then concatenates their declaration traces. If terms already stored numeric signature positions, those positions would have to be weakened and reindexed after every such concatenation.

With a family, the translator instead constructs a structural identity $p : S(d)$ immediately. Recursive results concatenate an occurrence trace of existential packages (d, p), while already constructed terms remain unchanged. For flattened defunctionalization, S is an indexed inductive family whose constructors represent source constants, application symbols, closures, and certified primitives. Each constructor's result index is its exact declaration; Section 4.2 spells out those four shapes.

From abstract identities to concrete symbols. The abstraction is removed only at a boundary where allocation information is available. The library provides a total structural conversion to a finite signature Ξ. Write $\mathsf{Symbol}(\Xi, d)$ for the type of positions in Ξ that select declaration d. A typed resolver has one component

$$r_d : S(d) \longrightarrow \mathsf{Symbol}(\Xi, d) \qquad \text{for every } d \in \mathsf{Decl}.$$

The proved raw-SMT route can also encode family terms directly. Its encoding record similarly supplies one naming component

$$\mathsf{name}_d : S(d) \longrightarrow \mathsf{String} \qquad \text{for every } d \in \mathsf{Decl},$$

with injectivity both between declarations and between two inhabitants of the same fiber, plus freshness from logical built-ins. For each traced pair (d, p), the command encoder emits the argument and result sorts from d and the concrete identifier $\mathsf{name}(p)$. Thus delaying allocation loses neither typing nor symbol identity. [8]

Definition 3.5 (Target model)**.** *A target model M_t supplies a nonempty carrier $[\![\kappa]\!]_{M_t}$ for each target sort. A declaration denotes the curried semantic type*

$$[\![d]\!]_{M_t} = [\![\kappa_1]\!]_{M_t} \to \cdots \to [\![\kappa_n]\!]_{M_t} \to [\![\kappa]\!]_{M_t}$$

for $d = (\kappa_1, \ldots, \kappa_n; \kappa)$. The model then supplies an interpretation component for every declaration:

$$I^d_{M_t} : S(d) \longrightarrow [\![d]\!]_{M_t} \qquad \textit{for every } d \in \mathsf{Decl}.$$

[7] Lean formalization: `Crush/Metatheory/FO/Family.lean and FO/FamilySemantics.lean`.

[8] Lean formalization: `Crush/Metatheory/FO/Family.lean, Defunctionalization/TranslationResult.lean, and SMT/Representation.lean`.

Hence two inhabitants of the same $S(d)$ may receive different interpretations, while every interpretation necessarily has the type prescribed by d. Target term denotation feeds the denotations of the typed argument vector to $I^d_{M_t}(p)$. The satisfaction relations $M_t \models \varphi$ and $M_t \models^{\mathsf{T}} T$ are then the standard many-sorted first-order semantics. [9]

Definition 3.6 (Target-theory unsatisfiability)**.** *A typed first-order theory is semantically unsatisfiable when it has no target model:*

$$\mathsf{Unsat}_{\mathsf{FO}}(T) \quad \Longleftrightarrow \quad \forall M_t.\, \neg(M_t \models^{\mathsf{T}} T).$$

4 Total flattened defunctionalization

Contents of this section. Arrow telescopes, closures, generated symbols, translation results, and the total algorithm.

4.1 Flattened arrow shape

Definition 4.1 (Arrow telescope)**.** *Every source type has a list of leading argument types* $\mathsf{args}(\tau)$ *and a non-arrow residual type* $\mathsf{res}(\tau)$*:*

$$\begin{aligned} \mathsf{args}(\mathsf{Bool}) &= [], & \mathsf{res}(\mathsf{Bool}) &= \mathsf{Bool}, \\ \mathsf{args}(\mathsf{Base}(\beta)) &= [], & \mathsf{res}(\mathsf{Base}(\beta)) &= \mathsf{Base}(\beta), \\ \mathsf{args}(\tau \to \sigma) &= \tau :: \mathsf{args}(\sigma), & \mathsf{res}(\tau \to \sigma) &= \mathsf{res}(\sigma). \end{aligned}$$

For example, if $\alpha = \tau_1 \to \tau_2 \to \upsilon$ and υ is non-arrow, then $\mathsf{args}(\alpha) = [\tau_1, \tau_2]$ and $\mathsf{res}(\alpha) = \upsilon$.

The indices of a target argument spine record that applying the listed arguments changes a source type α into a residual type γ. A *ground-result witness* proves that γ is Boolean or a base type; only then can the spine be emitted as one complete first-order symbol application. This indexed representation is what makes under-application impossible in the total translator. [10]

4.2 Captures, closures, and symbols

Definition 4.2 (Exact captures)**.** $\mathsf{FV}(e)$ *is the duplicate-free, left-to-right list of free de Bruijn positions in e. Beneath a binder, position zero is removed and positive positions are shifted down. A closure descriptor*

$$\chi = (\Gamma, \tau, \sigma, e), \qquad \Sigma; \Gamma, \tau \vdash e : \sigma,$$

stores the lambda's context, domain, codomain, body, and the typed references selected from $\mathsf{FV}(e)$*. If their types are $\gamma_1, \ldots, \gamma_m$, these are the exact constructor captures.* [11]

Definition 4.3 (Flattened symbol family)**.** *The target symbol family has four semantic roles:*

1. *a source constant $c : \alpha$, declared with arguments $\lfloor \mathsf{args}(\alpha) \rfloor$ and result $\lfloor \mathsf{res}(\alpha) \rfloor$;*

2. *an application symbol for every arrow $\tau \to \sigma$, declared as*

$$\mathsf{app}_{\tau,\sigma} : \mathsf{Fn}(\tau, \sigma) \times \lfloor \mathsf{args}(\tau \to \sigma) \rfloor \longrightarrow \lfloor \mathsf{res}(\tau \to \sigma) \rfloor;$$

[9] Lean formalization: `Crush/Metatheory/FO/FamilySemantics.lean`.

[10] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Spine.lean`.

[11] Lean formalization: `Crush/Metatheory/Defunctionalization/Collect.lean` and `Defunctionalization/Annotate.lean`.

3. a closure constructor

$$\mathsf{clos}_\chi : \lfloor\gamma_1\rfloor \times \cdots \times \lfloor\gamma_m\rfloor \longrightarrow \mathsf{Fn}(\tau, \sigma);$$

4. a certified primitive symbol carrying a proof that its chosen semantic value equals the canonical flattened interpretation of its source constant.

The roles remain distinct even when two declarations have the same shape. [12]

4.3 What translation returns

Definition 4.4 (Translation result)**.** *For $\Sigma; \Gamma \vdash e : \tau$, the total translation result $\mathcal{F}[\![e]\!]$ contains*

$$\mathcal{F}[\![e]\!] = (t_e, D_e, E_e, G_e, X_e, P_e),$$

where:

- *t_e is a target term in context $\lfloor\Gamma\rfloor^\star$ and sort $\lfloor\tau\rfloor$;*
- *D_e is the ordered trace of required symbol declarations;*
- *E_e is the list of closure equations;*
- *G_e is the list of representation guards;*
- *X_e is the list of extensionality formulas;*
- *P_e is the list of certified primitive constraints.*

The generated auxiliary theory and the complete theory of a source sentence are

$$\mathsf{Gen}(e) = E_e +\!\!+ G_e +\!\!+ X_e +\!\!+ P_e, \qquad \mathsf{Theory}(\varphi) = \mathsf{Gen}(\varphi) +\!\!+ [t_\varphi].$$

Lists are kept separate in Lean to retain provenance, then concatenated in this fixed order for semantic statements. [13]

4.4 How terms are translated

Boolean constructors, ground variables, ground constants, equality at ground types, and quantifiers translate structurally. Three cases require additional machinery.

Complete applications. The translator collects the left-associated source application spine. When its result is ground, the typed argument indices prove that all leading arrow arguments are present, so it emits exactly one flattened source or application symbol.

Residual function values. A variable, constant, or partial application of arrow type cannot be emitted as an under-applied first-order symbol. It is eta-expanded into a closure. The `LambdaBody` structure opens the complete leading lambda telescope and the theorem `denote_etaBody` proves that the resulting source function has exactly the original denotation. [14]

[12] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Symbol.lean`.

[13] Lean formalization: `Crush/Metatheory/Defunctionalization/TranslationResult.lean`.

[14] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Lambda.lean` and `Defunctionalization/EtaCorrectness.lean`.

Function equality. Equality between opaque function-sort values would be stronger than pointwise equality unless extensionality is asserted. For each function equality used by the translation, X_e receives a closed formula saying that pointwise equality over the complete arrow telescope implies equality of the function values.

Definition 4.5 (Flattened closure equation)**.** *Suppose χ has arrow type $\alpha = \tau_1 \to \cdots \to \tau_n \to \upsilon$ with ground υ, exact captures $\bar{y}$, and eta-opened body b. Its defining equation has the shape*

$$\forall \bar{y}\, x_1 \cdots x_n. \quad \mathsf{app}_\alpha(\mathsf{clos}_\chi(\bar{y}), x_1, \ldots, x_n) = t_b,$$

where app_α abbreviates the application symbol selected by the full arrow type α. The left side uses one flattened application, not a chain of unary applications. The right side carries all obligations recursively generated while translating b.

The three mutually recursive modes—ordinary term translation, application spine translation, and lambda-body translation—use a lexicographic measure built from constructor count and a mode tag. Lean therefore accepts them as total definitions, with no `partial` escape hatch. [15]

5 The canonical model and flattened correctness

Contents of this section. Why the model is chosen; term preservation; generated validity; unsatisfiability reflection.

5.1 Modeling function values

An arbitrary first-order target model may interpret $\mathsf{Fn}(\tau, \sigma)$ by any nonempty carrier. To prove soundness, however, it suffices to construct one target model from each source model. This is the role of the canonical model.

Definition 5.1 (Canonical flattened model)**.** *Given a source model M_s, its canonical target model $\widehat{M_s}$ uses the same carriers at Boolean and base sorts and uses the genuine source function space at each function sort:*

$$[\![\mathsf{Fn}(\tau, \sigma)]\!]_{\widehat{M_s}} = [\![\tau \to \sigma]\!]_{M_s}.$$

More generally, the Lean development exposes mutually inverse, type-directed maps $\mathsf{toCanonical}_\tau$ and $\mathsf{fromCanonical}_\tau$ so statements remain uniform at all types. In $\widehat{M_s}$:

- *flattened source symbols perform repeated source application;*
- $\mathsf{app}_{\tau,\sigma}$ *performs repeated source application;*
- clos_χ *reconstructs the exact captured source environment and denotes the source lambda;*
- *certified primitives use their proved canonical interpretation.*

This does not claim that all target models contain actual functions. It is a countermodel construction used only in the soundness proof. [16]

[15] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Translate.lean`.

[16] Lean formalization: `Crush/Metatheory/Defunctionalization/ModelExtension.lean` and `Defunctionalization/Flattened/Symbol.lean`.

A source valuation ρ induces a canonical target valuation $\widehat{\rho}$ by applying $\mathsf{toCanonical}$ pointwise.

Theorem 5.2 (Flattened term preservation)**.** *For every source model M_s, valuation ρ, and typed source term $\Sigma; \Gamma \vdash e : \tau$,*

$$[\![t_e]\!]_{\widehat{M}_s, \widehat{\rho}} = \mathsf{toCanonical}_\tau\big([\![e]\!]_{M_s,\rho}\big), \qquad \textit{where } \mathcal{F}[\![e]\!] = (t_e, D_e, E_e, G_e, X_e, P_e).$$

17

Proof architecture. One structural induction proves ordinary translation and spine translation simultaneously. Ground syntax is homomorphic. Complete spines use the currying theorem for the flattened symbol. Residual arrows use eta-closure correctness. Quantifiers use the fact that extending a canonical target valuation corresponds to extending the source valuation. □

Theorem 5.3 (Validity of generated obligations)**.** *For every e, the same canonical model satisfies the entire auxiliary theory:*

$$\widehat{M}_s \models^{\mathsf{T}} \mathsf{Gen}(e).$$

18

Proof architecture. A second simultaneous induction follows the three translator modes. Closure equations use term preservation and eta correctness; function extensionality is valid because the canonical function carriers are genuine source function spaces. Guards and primitive constraints are retained in the induction even when a particular structural term generates none. □

Theorem 5.4 (Flattened model extension)**.** *For every source sentence φ and source model M_s,*

$$M_s \models \varphi \quad \Longrightarrow \quad \widehat{M}_s \models^{\mathsf{T}} \mathsf{Theory}(\varphi).$$

19

Proof. Generated-obligation validity proves the prefix $\mathsf{Gen}(\varphi)$. Flattened term preservation at the empty valuation proves the final assertion t_φ. □

Corollary 5.5 (Flattened unsatisfiability reflection)**.**

$$\mathsf{Unsat}_{\mathsf{FO}}(\mathsf{Theory}(\varphi)) \quad \Longrightarrow \quad \mathsf{Unsat}_{\mathsf{HO}}(\varphi).$$

20

Proof. If φ had a source model, model extension would construct a target model of $\mathsf{Theory}(\varphi)$, contradicting the premise. □

[17] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Denotation.lean`.
[18] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Theory.lean`.
[19] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Theory.lean`.
[20] Lean formalization: `Crush/Metatheory/Defunctionalization/Flattened/Theory.lean`.

5.2 The unary reference proof

The development retains a smaller classic translation $\mathcal{D}[\![e]\!]$ that uses a binary application symbol at each arrow. A source/target logical relation is defined at Boolean, base, and arrow types; its arrow clause says that related function values produce related results when observed through unary application. The fundamental lemma proves

$$[\![e]\!]_{M_s,\rho_s} \approx_\tau [\![\mathcal{D}[\![e]\!]]\!]_{M_t,\rho_t}$$

under related models and valuations. The flattened currying development then proves that the production-shaped translation and the unary reference have the same canonical denotation. Thus the unary proof is an independent semantic reference, not the endpoint of the current VCG theorem. [21]

6 From typed FO theories to raw SMT commands

Contents of this section. Why raw SMT semantics is relational; encoding choices; exact representation; semantic model lifting.

The production SMT syntax is intentionally not intrinsically typed. The proof therefore does not pretend that every raw term is well sorted. Instead it defines a semantic relation for the fragment emitted by the proved encoder.

Definition 6.1 (Raw SMT model)**.** *A raw model M_q contains one universe of values, a membership predicate* $\mathsf{inSort}(s, v)$*, nonemptiness of every concrete sort, distinguished Boolean values, literal interpretations, and a graph* $\mathsf{apply}(p, \bar{v}, w)$ *for user symbols. A declaration requires the graph to be total, single-valued, and sort-correct at that declared type. An inductive evaluation relation gives Boolean connectives, equality, binders, and user-symbol applications their intended meanings.* [22]

Only the command forms needed by this path are semantically admitted: sort and function declarations, assertions, and administrative/query commands. Definitions, recursive definitions, lambdas, and datatype declarations are outside the verified raw-command fragment.

Definition 6.2 (Semantic command unsatisfiability)**.** *$M_q \models_{\mathsf{SMT}} C$ means that M_q satisfies every supported command in the ordered array C. Define*

$$\mathsf{Unsat}_{\mathsf{SMT}}(C) \quad \Longleftrightarrow \quad \forall M_q.\, \neg(M_q \models_{\mathsf{SMT}} C).$$

This is a mathematical property of commands, not a claim about a solver process or a reported result.

Definition 6.3 (Encoding choice)**.** *For a typed symbol family S, an encoding $\mathcal{E}$ chooses:*

- *a concrete SMT sort $\mathcal{E}_s(\kappa)$ for every FO sort, injectively, with* Bool *mapped to SMT* `Bool`*;*
- *a concrete name $\mathcal{E}_n(p)$ for every typed symbol, injectively across both symbol identity and declaration shape;*
- *proofs that generated names do not collide with logical built-ins.*

[21] Lean formalization: `Crush/Metatheory/Defunctionalization/Fundamental.lean`, `Defunctionalization/ModelExtension.lean`, and `Defunctionalization/Flattened/Currying.lean`.

[22] Lean formalization: `Crush/Metatheory/SMT/Semantics.lean`.

The pure term encoder $\mathcal{A}_{\mathcal{E}}[\![t]\!]$ maps variables to numeric de Bruijn indices, symbols to their chosen names, and logical constructors to their corresponding raw SMT forms. [23]

Definition 6.4 (Exact theory representation)**.** *Given an ordered declaration trace D and typed theory T, the pure encoder first emits required sort declarations, then symbol declarations, then one assertion per formula in T, preserving stable order. Write*

$$\mathsf{Rep}(\mathcal{E}, T, C)$$

when there exists an explicit ordered declaration trace D such that C is definitionally equal to that pure encoding. For a source sentence, define

$$\mathsf{Commands}(\mathcal{E}, \varphi) = \mathsf{encode}_{\mathcal{E}}(D_{\varphi}, \mathsf{Theory}(\varphi)).$$

The theorem `encode_translation` *proves* $\mathsf{Rep}(\mathcal{E}, \mathsf{Theory}(\varphi), \mathsf{Commands}(\mathcal{E}, \varphi))$ *by construction.*

Theorem 6.5 (FO-to-SMT representation soundness)**.** *If* $\mathsf{Rep}(\mathcal{E}, T, C)$ *and* $M_t \models^{\mathsf{T}} T$*, then there is a raw SMT model M_q such that $M_q \models_{\mathsf{SMT}} C$.* [24]

Model construction. The raw universe tags values by their intrinsic target sort. Sort membership checks the tag. Each encoded user-symbol graph decodes tagged arguments, applies the corresponding typed target interpretation, and re-tags the result. Injective sort and name maps prevent accidental aliasing. A term-evaluation lemma relates raw evaluation of $\mathcal{A}_{\mathcal{E}}[\![t]\!]$ to intrinsic denotation of t; declaration and assertion lemmas then establish the complete command sequence. □

Corollary 6.6 (SMT command unsatisfiability reflects to FO)**.** *If* $\mathsf{Rep}(\mathcal{E}, T, C)$*, then*

$$\mathsf{Unsat}_{\mathsf{SMT}}(C) \Longrightarrow \mathsf{Unsat}_{\mathsf{FO}}(T).$$

[25]

7 Lean reification and the proved VCG route

Contents of this section. Structural witnesses; pure and stateful command generation; the composed theorem; exact scope.

7.1 The supported Lean boundary

Definition 7.1 (Reification witness)**.** *The executable reifier recognizes supported normalized Lean-expression constructors and returns an existentially typed source term. A witness records:*

- *the inferred Lean type and the expected retained Lean type;*
- *exact correspondence of recognized expression and intrinsic-term shapes;*
- *the typed local-context bridge;*
- *the typed finite signature bridge for global constants.*

[23] Lean formalization: `Crush/Metatheory/SMT/Representation.lean`.
[24] Lean formalization: `Crush/Metatheory/SMT/Soundness.lean`.
[25] Lean formalization: `Crush/Metatheory/SMT/Soundness.lean`.

For a closed proposition, successful reification packages a source signature Σ, a sentence φ, and this witness. [26]

The witness is structural. It does not define a denotation for arbitrary Lean expressions and therefore does not, by itself, prove that Lean's proposition and φ have the same truth value. Unsupported syntax causes reification to return no result rather than entering the proved route.

7.2 Pure and stateful VCG

Definition 7.2 (Proved VCG commands)**.** *For encoding $\mathcal{E}$ and source sentence φ, the pure VCG function computes $\mathcal{F}[\![\varphi]\!]$ and then the exact command array* $\mathsf{Commands}(\mathcal{E}, \varphi)$*. Its result retains the theorem*

$$\mathsf{Rep}(\mathcal{E}, \mathsf{Theory}(\varphi), \mathsf{Commands}(\mathcal{E}, \varphi)).$$

[27]

The total stateful function `VCG.run` starts from fresh translation bookkeeping and installs exactly this array in `TranslateState.commands`. The theorem `run_represents` proves both command equality and the representation property. Fresh state matters: the proved result cannot inherit commands or trust markers from a previous direct translation. [28]

Theorem 7.3 (Composed VCG soundness)**.** *Let C be the exact command array in the fresh state returned by `VCG.run` for $\mathcal{E}$ and φ. Then*

$$\mathsf{Unsat}_{\mathsf{SMT}}(C) \Longrightarrow \mathsf{Unsat}_{\mathsf{FO}}(\mathsf{Theory}(\varphi)) \Longrightarrow \mathsf{Unsat}_{\mathsf{HO}}(\varphi).$$

[29]

Proof. The state representation theorem supplies $\mathsf{Rep}(\mathcal{E}, \mathsf{Theory}(\varphi), C)$. SMT representation soundness reflects command unsatisfiability to the typed FO theory. Flattened unsatisfiability reflection then yields source unsatisfiability. □

Corollary 7.4 (Structural Lean-facing specialization)**.** *If a closed Lean expression has a reification witness for φ, the same command-unsatisfiability premise implies* $\mathsf{Unsat}_{\mathsf{HO}}(\varphi)$*. This is theorem `encoded_unsat_implies_source_unsat`. Its conclusion is intentionally about φ, not about a postulated semantics of the original `Lean.Expr`.*

7.3 Proved versus trusted status

`VCG.TranslationStatus` makes the boundary structural. A `proved` result stores the intrinsic sentence, exact commands, and their representation theorem. A `trusted` result stores commands and a nonempty list of trust reasons but exposes no theorem for reflecting command unsatisfiability. The legacy extensible `emitTerm` route is marked trusted at its root; unrestricted handlers, uncertified constants or closures, and native-HO emission add more specific reasons. [30]

[26] Lean formalization: `Crush/Metatheory/Reification/Type.lean, Reification/Term.lean, Reification/Capture.lean, Reification/Reify.lean, and Reification/Witness.lean`.
[27] Lean formalization: `Crush/Metatheory/VCG/Generate.lean`.
[28] Lean formalization: `Crush/Metatheory/VCG/Stateful.lean`.
[29] Lean formalization: `Crush/Metatheory/VCG/Soundness.lean`.
[30] Lean formalization: `Crush/Metatheory/VCG/Trust.lean and VCG/Status.lean`.

8 Interpreted operations and guarded carriers

Contents of this section. What is modeled now; semantic contracts for extensions; limitations of the proved command fragment.

Definition 8.1 (Guarded carrier encoding)**.** *A guarded encoding of a source carrier S in a target carrier T consists of an encoder $\epsilon : S \to T$, a guard $G : T \to \mathsf{Prop}$, and a decoder $\delta : (t : T) \to G(t) \to S$ satisfying*

$$G(\epsilon(s)), \qquad \delta(\epsilon(s)) = s, \qquad G(t) \Rightarrow \epsilon(\delta(t)) = t.$$

Thus S is equivalent to the guarded subtype of T. Universal binders become guarded implications, existential binders become guarded conjunctions, and equality is reflected by injectivity. The canonical example represents $\mathbb{N}$ in $\mathbb{Z}$ with guard $0 \leq z$. [31]

Certified hook contracts tie a source constant to a target interpretation by a semantic equality proof. These contracts avoid axioms: a primitive is certified only when its chosen target meaning is proved equal to the canonical meaning of its source constant. Arbitrary metaprogram handlers remain trusted. [32]

Guarded encodings and certified primitives are modeled components, but the current proved raw-command encoder treats flattened symbols as fresh user symbols. A full proved path for interpreted integer operations requires an extended encoding and model-lifting theorem for the corresponding SMT built-ins; it is roadmap work, not an implicit consequence of the present theorem.

9 Proof roadmap and trust boundary

Contents of this section. Completed chain; exact non-claims; prioritized future work.

Figure 1 shows the completed semantic chain. Every solid arrow is backed by a Lean theorem.

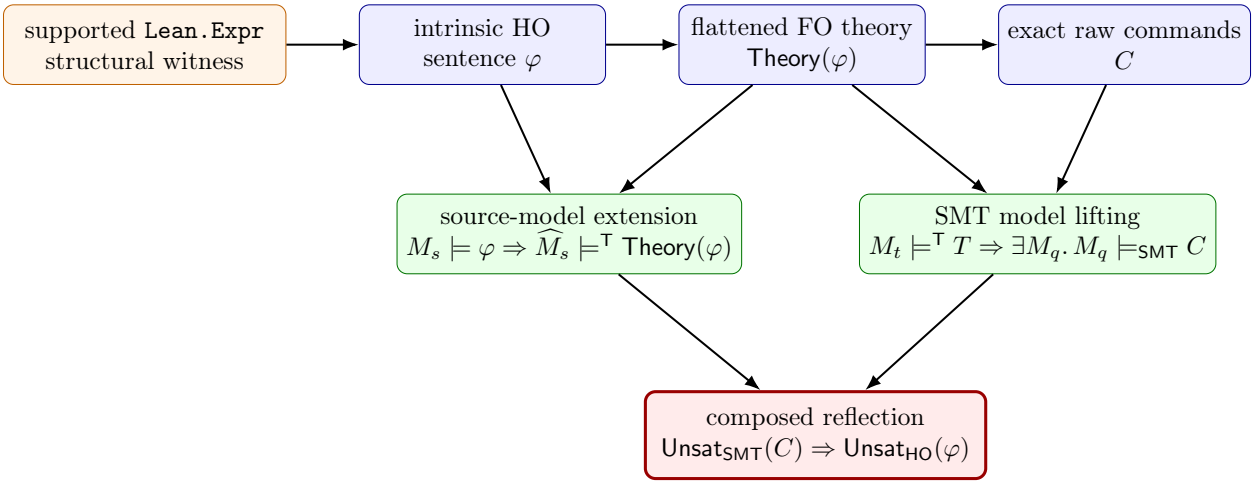

Figure 1: The completed proof route. The top arrows are, from left to right, partial reification, total flattened translation $\mathcal{F}$, and exact SMT encoding. The leftmost box is structural rather than denotational; all semantic conclusions are stated about the reified HO sentence.

[31] Lean formalization: `Crush/Metatheory/Guarded/Encoding.lean`.
[32] Lean formalization: `Crush/Metatheory/Hooks.lean`.

9.1 What is proved today

1. The HO and FO syntax is intrinsically typed, and the flattened defunctionalizer is total on every constructor of the modeled core.

2. Exact captures, complete application spines, eta closures, and function extensionality are modeled in the production-shaped flattened theory.

3. Translated terms preserve denotation in the canonical model, and every generated obligation is valid in that same model.

4. Every source model extends to a model of the complete flattened theory; hence target-theory unsatisfiability reflects to source unsatisfiability.

5. The pure encoder produces an exact ordered representation in raw SMT syntax, and every typed target model lifts to a model of those commands.

6. Fresh stateful VCG execution retains the exact representation theorem and composes both reflection results.

7. Supported Lean expressions retain typed structural reification evidence; proved and trusted whole-translation results are distinct constructors.

9.2 What is deliberately not claimed

- No denotational semantics is postulated for arbitrary `Lean.Expr`; the final proposition concerns the reified intrinsic sentence.

- Semantic command unsatisfiability is a premise. Correctness of an external solver's reported `unsat` answer requires certificate replay or another independently checked argument.

- The legacy direct, extensible `emitTerm` pipeline does not acquire a whole-run theorem merely because some local closures or primitives are certified.

- Unsupported syntax, dependent function types, datatypes in the proved raw-command fragment, unrestricted translation handlers, and native-HO commands are outside the present proved route.

9.3 Prioritized roadmap

1. Connect the proved route to user-facing execution.
Route supported production goals through `reifySentence?` and `VCG.run`, retain the resulting `proved` status through solver invocation, and make fallback to the direct translator explicit and observable.

2. Strengthen the Lean boundary semantically.
Replace the current constructor/type correspondence with a theorem connecting the meaning of the supported Lean proposition to the reified HO sentence. This is the missing step needed to state the final result directly about Lean's proposition rather than only about φ.

3. Certify interpreted theories.
Extend the raw SMT semantics, encoding record, and model-lifting proof for interpreted integers and other selected built-ins. Compose guarded carrier encodings such as $\mathbb{N} \hookrightarrow \mathbb{Z}$ with those representations.

4. **Expand structural coverage.**
   Add proof-facing datatype encodings, recursive datatype declarations, and selected value-indexed dependent forms. Each addition requires syntax, semantics, total reification, translation, and command representation—not only a new lowering rule.

5. **Integrate certified hooks compositionally.**
   Give certified head/family translations a representation theorem that can be combined with structural translation without granting semantic power to unrestricted handlers.

6. **Close the solver-result boundary.**
   Connect accepted Alethe replay or another kernel-checked certificate format to the premise that the commands are semantically unsatisfiable. Keep fast trusted solver mode explicitly separate.

10 Machine-checked theorem inventory

Contents of this section. The named results that implement the proof story.

The principal declarations, in proof order, are:

1. `flattenArrow_eq`, in namespace `Flattened.TargetArguments`: related spine lemmas show that indexed spines match the flattened arrow telescope.
2. `Flattened.LambdaBody.denote_etaBody`: opening the full lambda telescope preserves the source function denotation.
3. `Flattened.TargetArguments.completeApp_denote`: one flattened application symbol denotes repeated source application.
4. `Flattened.translate_denote`: the translated term component has the source denotation in the canonical model.
5. `Flattened.generated_valid`: all generated equations, guards, extensionality formulas, and primitive constraints are valid.
6. `Flattened.model_extension`: every satisfying source model extends to the complete flattened target theory.
7. `Flattened.target_unsat_implies_source_unsat`: FO unsatisfiability reflects to the HO sentence.
8. `SMT.encode_translation`: the pure encoder exactly represents the complete flattened theory.
9. `SMT.representation_sound`: every represented FO model induces a model of the exact raw commands.
10. `SMT.commands_unsat_implies_theory_unsat`: semantic raw-command unsatisfiability reflects to typed FO unsatisfiability.
11. `VCG.run_represents`: the fresh state returned by the total VCG contains the exact represented commands.
12. `VCG.run_unsat_implies_source_unsat`: the direct composition for stateful VCG.

13. `VCG.encoded_unsat_implies_source_unsat`: the specialization retaining a structural witness for the original Lean expression.

The earlier unary results—`fundamental`, `fundamental_sentence`, and the unary `target_unsat_implies_source_unsat`—remain machine-checked as an independent reference model.